

Online Indoor Fire Evacuation System

An-Fong Lee  and Pei-Hsuan Tsai , *Member, IEEE*

Abstract—Many methods for indoor fire escape evacuation have been proposed to assist victims in escaping. Greedy approaches focus on finding the closest exit and providing victims with the shortest evacuation path. However, they are time consuming to respond to a dynamic fire spread. Optimal approaches determine evacuation exits and routes based on the characteristics of indoor fires using probabilistic or gravitational algorithms. However, their disadvantage is the excessive amount of computation required. Compared with previous methods, this article provides a lightweight exit decision formulation paired with dynamic path planning. The experimental results indicate that our approach accelerates the computation time by approximately 1.5 times and achieves a victim survival rate higher than 90%.

Index Terms—D* Lite, indoor fire evacuation, navigation, path planning.

I. INTRODUCTION

INDOOR fires cause thousands of deaths and millions of dollars in property damage each year in the United States [1]. When an indoor fire occurs, the indoor temperature increases rapidly and a large amount of smoke with toxic gases fills the indoor space, causing limited and short effective escape time in the fire [2], [21]. However, the invisibility of the indoor fire causes the victims to lose their way, resulting in panic and overwhelming behavior. These emotional behaviors lead to the waste of escape time. Therefore, providing online and real-time guidance for victims of fires is an effective solution.

Many fire evacuation systems have been developed to help victims escape. Majority of the systems rely on centralized computing, in which the server determines the evacuation route for each victim separately. However, when the network is disrupted, the client cannot receive any escape information in real time, including maps, environmental data, and updated evacuation routes. Clients can rely only on outdated information to assist victims in escaping, which may be incorrect and dangerous.

Another type of distributed fire evacuation system uses a sensor network to obtain the environmental data. Sensors periodically broadcast sensing data. The embedded device of users

receives sensing data within its communication range and then calculates the evacuation path. This type of system is more robust, real-time, and scalable because it does not rely heavily on the network, and each device optimizes the escape route in real time based on its location and sensor information. However, owing to the limited capabilities of embedded devices, an escape algorithm with a lower computing cost is required. In addition, embedded devices can only obtain partial environmental data owing to the limited communication range of a sensor, and an on-the-fly update mechanism is required to adapt to changes in the received sensing data.

A general type of evacuation algorithm computes the shortest paths to all exits and then determines the closest exit for the victim [3], [4]. However, it is time consuming to be used online. Another type of algorithm first determines the designated exit and then computes the path to the designated exit [5]. However, their exit selection approach is still time consuming; therefore, their evacuation paths cannot be updated in real time. Hence, this article aims to provide a low-cost computation with fast updating to support online evacuation for stand-alone embedded devices. This article has two critical objectives. One is the real-time determination of the safest exit, and the other is the real-time update of the safest path.

For clarity, the contributions of this article are summarized as follows:

- 1) A complete procedure for a more adaptable and robust distributed fire evacuation system is proposed, including the settings of indoor map preprocessing for the generation of evacuation paths. In particular, this article designed a lightweight indoor fire escape algorithm for small personal mobile devices in a distributed fire evacuation system.
- 2) A speedup safest exits selection algorithm is presented in this article. The results of the experiments demonstrate that it only takes less than 0.1 ms to compute the safest escape exit for a victim in an indoor floor map with ten exits.
- 3) An improvement in the D* dynamic path planning algorithm is proposed to make the evacuation path safer for indoor fire scenes, thereby helping trapped victims escape the drastically changing indoor fire environments.

The remainder of this article is organized as follows. Section II introduces related fire-evacuation approaches. In Section III, we present the design of the safest evacuation path planning (SEPP) method. Section IV describes the safest pathfinder (SAF) approach suitable for indoor fire evacuation. Section V presents the experimental results combined with an fire dynamics simulator (FDS) fire scene simulation. Finally, Section VI presents conclusions and discusses the direction of future research.

Manuscript received 1 May 2022; revised 21 September 2022 and 2 December 2022; accepted 10 January 2023. Date of publication 7 February 2023; date of current version 30 August 2023. This work was supported in part by the Ministry of Science and Technology of Taiwan under Grant MOST 110-2221-E-006-008; and in part by the Higher Education Sprout Project, Ministry of Education to the Headquarters of University Advancement, National Cheng Kung University. (Corresponding author: Pei-Hsuan Tsai.)

The authors are with the Institute of Manufacturing Information and Systems, National Cheng Kung University, Tainan 70101, Taiwan (e-mail: p96094155@gs.ncku.edu.tw; peihsuan.tsai@gmail.com, phtsai@mail.ncku.edu.tw).

Digital Object Identifier 10.1109/JSYST.2023.3238348

II. RELATED WORK

A. Safest Exit

The definition of the safest exit for a victim is that the exit is kept safe in fires until the victim passes through it. However, several approaches only consider the distance and location of an exit when selecting an exit, instead of safety. Liu et al. [6] estimated the victims' nearest escape exit using an A* algorithm. This is a feasible strategy during the early stages of an indoor fire, however the status of the exit may become dangerous with the spread of fire. Ye and Tsai [7] considered the distance and fire spread when selecting an exit. It applies the concept of universal gravitation such that the fire field generates a thrust vector, and the exit generates a pull vector. The trajectory was determined based on the direction of the calculation force. However, as time passes, the fire field expands outward, which causes the calculation of force to grow exponentially. To solve these problems, this article proposes a lightweight safest exit selection formula called real-time safest exit computation (RSEC) to quickly compute the safest escape exit based on both distance and risk factors.

B. Evacuation Path Planning

The definition of the safest path for a victim is that the path remains safe in fires until the victim goes through it. The further away from the fire, the safer the path. Owing to the rapid spread of fire, the evacuation path must be updated in real-time to ensure safe and timely evacuation. Existing path planning approaches fall into several categories, including sampling-based methods such as the rapid exploration random tree and probabilistic roadmap [8], [9]. They are unstable, and their results diverge each time. Therefore, other meta-heuristic algorithms such as GA and ant colony optimization (ACO) have been proposed [10], [11], [12]. In [11], the victim's escape path was generated using the ACO. By dividing the fire into three stages and matching the tuning parameters for the specific stages, ACO assists in making evacuation paths as close as possible to the actual fire environment. However, these meta-heuristics require a lot of trial and error to generate the paths; therefore, they cannot be used in dynamic scenes with high-frequency changes. Compared with the previous two methods, the graph search method (e.g., A*, LPA*, or Dijkstra) is more stable and efficient [6], [13], [14], [15]. Liu et al. [6] calculated the Manhattan distance to determine the closest exit to the victim and then determined the path using the A* algorithm. When an indoor fire disrupts the evacuation path, the server must replan the entire evacuation path for all the victims. However, because the indoor environment frequently changes [6], it is inefficient. In addition, the shortest evacuation path may not be the safest.

C. Dynamic Path Planning

The D* algorithm is designed to explore unknown environments, and effective search and rapid repair capability planning algorithms have become increasingly prevalent. However, the D* algorithm is extremely complex in practice; consequently,

the original author announced in 2002 that the execution speed is equivalent to that of D* and that the more concise D* Lite algorithm must completely replace it.

D* Lite is an effective incremental path-planning algorithm that combines the effective search of heuristic search (e.g., A*) with the fast path repair capability of incremental search (e.g., Dynamics SWSF-FP). Koenig et al. [20] demonstrated the benefits of an efficient incremental path planning. As shown in Fig. 2(a), when efficient incremental path planning in the lower right corner is used to initiate the path, the number of grids to be explored is comparable to that of the heuristic search algorithm of A*. In addition, if the map changes while the robot is traveling to its destination, D* lite retains the characteristics of incremental search. During the search process, candidate solutions are placed in a priority queue, which consumes a small amount of storage space; however, when the optimal solution is identified, it is removed. When the solution is compromised, previously stored candidate solutions can be rapidly retrieved and implemented. As shown in Fig. 2(b), when the D* lite algorithm is affected by map changes, the number of search grids required is substantially lower than that required by the A* algorithm.

Because the D* Lite path-planning algorithm is highly adaptable to dynamic environments, it is suitable for use in fire scenes where the indoor environment changes frequently. However, the D* Lite algorithm is not built for fire escape scenarios; this article enhances the D* Lite algorithm by changing the calculation of keys to the Euclidean distance between the candidate path and the fire source, namely, advanced D* Lite. This improvement makes the escape route consider the path length and safety factor.

III. SYSTEM ARCHITECTURE

Here, the procedure for SEPP and the terms and notation used in later sections are introduced. The SEPP procedure, as shown in Fig. 1, comprises two phases: initialization and running. The initialization phase transforms a real-world indoor floor map into a graph, whereas the running phase updates the graph information and finds the safest evacuation path for the victims. Fig. 3 shows the execution timeline of the SEPP method. The map updater (MU) periodically checks the sensing data; when it is abnormal, the MU renews the graph. SAF then plans the safest evacuation path for the victim based on the renewed graph. By setting a short period, SAF is responsive to an indoor fire environment. The periodic task is recursively executed until the victim successfully escapes.

Procedure of SEPP

- *Step 1:* Gridding the indoor floor map into a grid map (GM) with optimal resolution.
- *Step 2:* Graphing the GM into a graph, except for grids that contain obstacles.
- *Step 3:* The MU updates the passable attributes of each node in the graph based on the sensing data.
- *Step 4:* The safest evacuation path is generated by performing RSEC and Advanced D* Lite of the SAF.

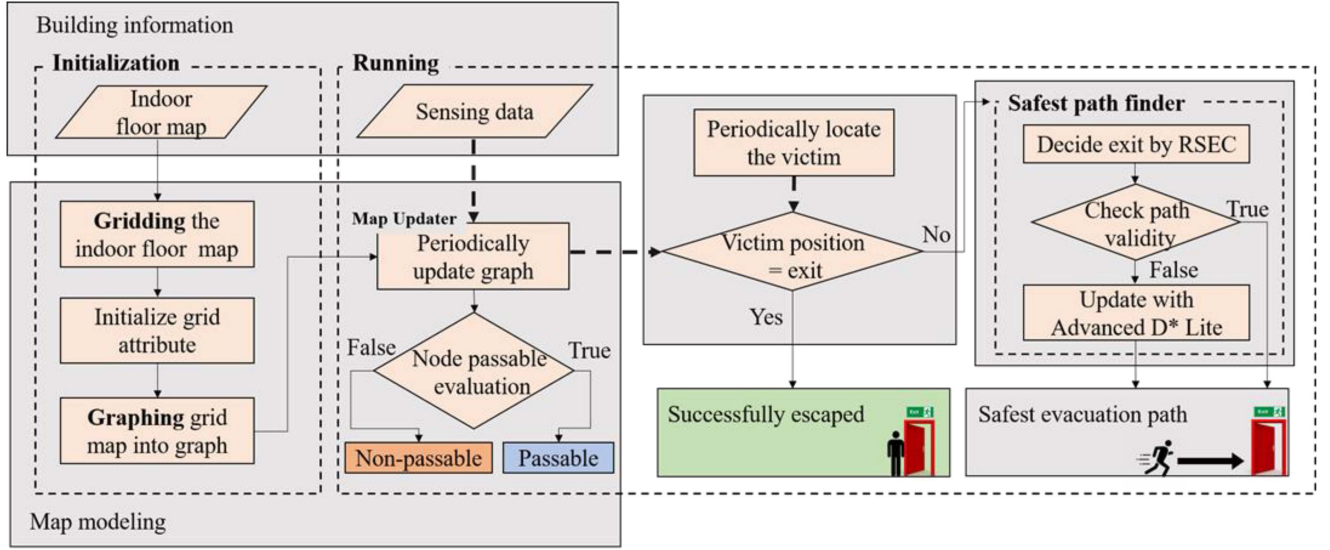


Fig. 1. Overview of the SEPP method.

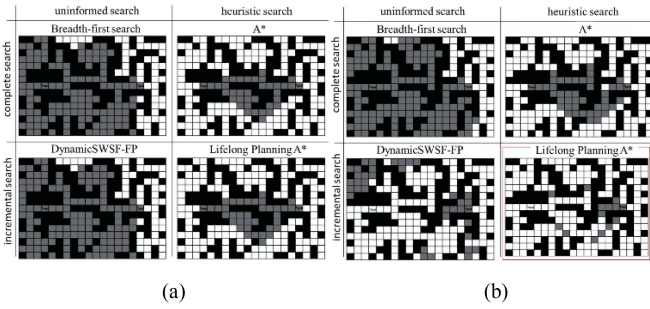


Fig. 2. (a) Efficient incremental path planning on initial search. (b) Efficient incremental path planning on follow-on search.

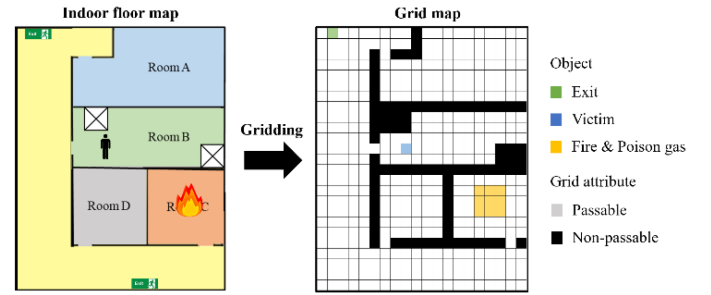


Fig. 4. Conversion of an indoor floor map into a GM.

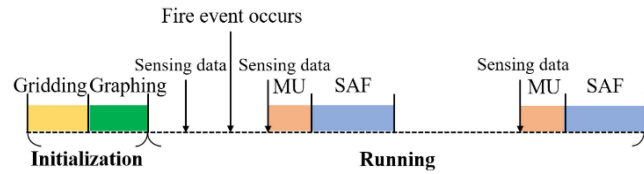


Fig. 3. Procedure of SEPP.

A. Initialization

The initialization phase (steps 1 and 2) of the SEPP procedure includes gridding and graphing, as shown in Fig. 3. Gridding transforms the indoor floor map into a GM with optimal resolution, whereas graphing converts the GM into a graph to simplify the number of elements in the graph.

1) *Gridding*: The gridding method converts indoor maps into GMs through digitization and virtualization, as shown in Fig. 4. Because the nongrid indoor map has a continuity angle between 0° and 360° for the victim, digitization can reduce the continuity angle to a maximum of eight directions, which significantly reduces the calculation cost during path planning. Grid (g_i) represents a square physical area, which is the most basic unit in the GM.

For most grid-based path planning algorithms, the calculation time is proportional to the grid resolution. Therefore, a low-resolution GM requires a shorter calculation time than a higher resolution one. However, the GM with a lower resolution results in a larger grid area, which may omit a passable evacuation path. For instance, when the resolution of a GM is low, a columnar obstacle in the center of the grid that the victim can easily bypass is considered the entire grid occupied, even if the obstacle only places a portion of the grid. The incorrect results of the grid classification caused the evacuation path to be omitted.

Fig. 5 shows the difference between low-resolution and high-resolution GMs. Evidently, the high-resolution GM is closer to the real-world indoor map than the lower one; however, the cons of higher resolution setting is longer computing time to obtain the evacuation path. This article presents an equation to strike a balance between both computation time and resolution. According to different application requirements for indoor fire evacuation, our equation computes the corresponding and most suitable resolutions. The resolution computation uses the following equation:

$$\text{BestSidelength} = \text{Min}(\text{Max}(\text{VictimWidth}(i)))$$

$$\forall i = \text{all kinds of victim.} \quad (1)$$

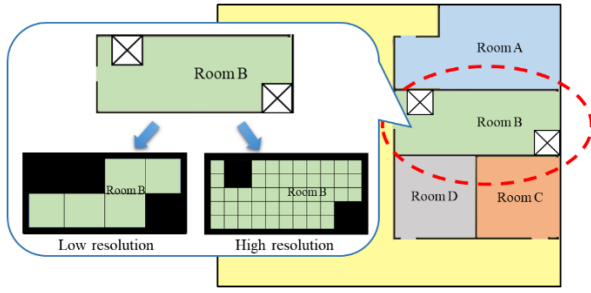


Fig. 5. Transformation of the indoor floor map into GMs with different resolutions.

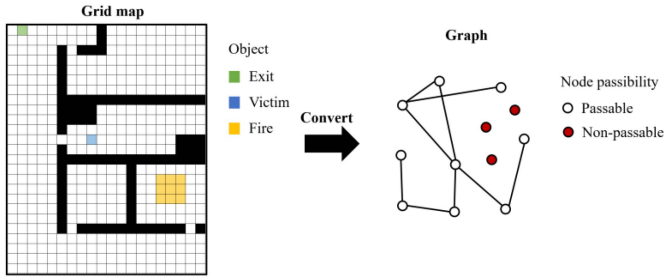


Fig. 6. Conversion of a GM into a graph.

The side length of the grid should be set to the minimum width of the various victims. For instance, in a hospital, a victim may sit on a wheelchair; therefore, the side length of the grid should be set as the longest side of the wheelchair. Using this setting, the grid search algorithm can obtain the most suitable GM according to different situations. All victims were able to escape path security based on SPF.

2) *Graphing*: Graphing is the process of transforming a GM into a graph. The advantage of graphing is to reduce the computing cost by integrating the nonobstacle grid of the GM while retaining the main elements of the indoor floor map. A graph of nodes (n_i) and edges (e_i) is composed, as shown in Fig. 6. Each node has a Boolean attribute called *passable*, defined as n_{pass} . Any two adjacent nodes are connected by an edge, which is defined as $e_i(n_i, n_j)$.

B. Runtime

The running phase is a periodic task that includes examining abnormal sensing values, updating graphs, and finding the safest path.

Sensing data, such as temperature, smoke, and toxic gases indicate the safety of an area within the communication range of the sensor. Consider the temperature as an example. When the indoor temperature exceeds 66 °C, the human body is burned and it is difficult to breathe [16]. These are defined as abnormal values in this article, and humans cannot pass through these dangerous subareas during evacuation.

The main task of the MU is to periodically update the passable attributes of the nodes in the graph based on sensing data. As shown in Fig. 7, the MU classifies the passable attribute of the nodes in the graph corresponding to each subarea into

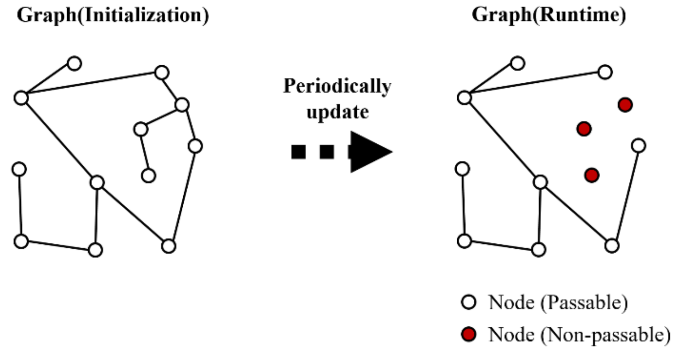


Fig. 7. Updating the attribute of the node in the graph.

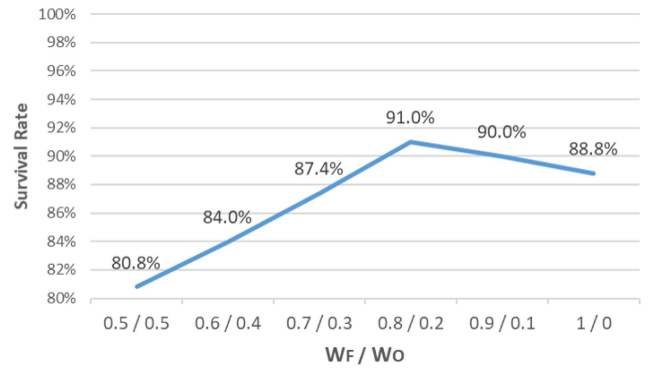


Fig. 8. Survival rate with increasing ratio of w_F/w_O .

“True,” or “False,” which are also known as the passable node and nonpassable node. The safest path is composed only of passable nodes. Finding the safest path consists of the safest exit computation and dynamic path planning; the details are described in the next section.

IV. SAFEST PATH FINDER

SAF verifies a valid and safe path for the victim periodically and mainly consists of two algorithms: RSEC and advanced D* lite. The Algorithm 1 describes the SAF procedure. The inputs are the graph (G) and the node of the victim (N_V). Output is the evacuation path of the victim (V_{SAP}) in the graph.

- *Step 1*: (Lines 2–4) Check the previous evacuation path to ensure that all nodes that the victim has not visited are still passable.
- *Step 2*: (Line 5) Initialize the set of exits (S_E) containing all passable exits in the graph.
- *Step 3*: (Lines 7–9) When all exits are destroyed, they remain in place and wait for rescue.
- *Step 4*: (Line 11) For each passable exit in the S_E , RSEC is performed to obtain the safest destination exit (E_D) for the victim.
- *Step 5*: (Line 12) Execute the Advanced D* Lite algorithm to obtain the safest evacuation path from the victims coordinates to the destination exit.
- *Step 6*: (Line 16) When the E_D is unreachable, it is removed from S_E .

Algorithm 1: Procedure of SAF.

Input: Graph, (G); Previous Safest Path, (P_P)
Output: Safest Path, (P)

1. Localize the victim's node, (V)
2. **if** CheckPreviousPath(G, V, P_P)
3. **return** P_P
4. **end if**
5. S_E = all the passable exits in G
6. P = NULL
7. **if** S_E is NULL **then**
8. **return** P
9. **end if**
10. **while** S_E not NULL **then**
11. E_D = RSEC(G, V, S_E)
12. P = **Advanced_D*_Lite**(G, V, E_D)
13. **if** P is not NULL **then**
14. **return** P
15. **end if**
16. remove E_D **from** the S_E
17. **end while**
18. **return** P

A. Real-Time Safest Exit Computation

The design strategy for selecting the safest escape exit is based on two perspectives. One considers the environmental delay, which is defined as the number of obstacles between the victim and the exit. The other is to consider the fire delay, which is the variance of the distance between the exit and victim affected by the speed of the victim and the spread of the fire. Each exit receives a risk score, R_{E_i} by formula

$$R_{E_i} = O_{\text{num}} \times w_O + F_{E_i} \times w_F \quad (2)$$

where O_{num} is the number of obstacle grids in the rectangle, where the endpoints of the diagonal line are the victim and the exit. F_{E_i} is the influencing factor derived from indoor fire development in [17]

$$F_{E_i} = \text{Risk}(x). \quad (3)$$

According to [17], the risk level of an indoor fire can be approximately obtained as $\text{Risk}(x) = e^{x-9}$, where x is the time elapsed since the fire event occurred. The victim can obtain the risk level corresponding to each exit by estimating the arrival time (T_{arr}) at the exit

$$T_{\text{arr}}(E_i) = \frac{D_{\text{EU}}(E_i, V)}{\text{Speed}_V} \quad (4)$$

where D_{EU} denotes the Euclidean distance between the exit (E_i) and victim (N_v).

w_F and w_O are the weights of F_{E_i} and O_{num} , respectively. In this article, w_F and w_O were experimentally determined. Based on the experimental results, we selected the proportion with the highest survival rate ($w_F/w_O = 0.8/0.2$). Fig. 8 shows the survival rate with respect to the increasing ratio of w_F to w_O . By sorting the risk score of each exit, the exit with the lowest risk score was set as the escape exit destination.

B. Advanced D Lite*

A path is a sequence of distinct nodes such that any two consecutive nodes are connected through an edge, and $p[n_0, n_k] = \langle n_0, n_1, \dots, n_{k-1}, n_k \rangle$ denotes one of the paths beginning from n_0 and ending at n_k . Let $n_i \in p[n_0, n_k]$ indicate that n_i is a node in $p[n_0, n_k]$. In an evacuation path, EP_i indicates that all node attributes in $p[n_0, n_k]$ are “passable.” The shortest evacuation path, $\text{SHP}_i[n_{i,0}, n_{i,k}]$, has the minimum number of nodes among all the evacuation paths.

The D* Lite algorithm [18] has been widely used to guide mobile robots by a computed path from the start node to the destination node in an unknown terrain. The evacuation path (EP_i) comprises a set of nodes in which all passable attributes (n_{pass}) are True. Each node has a *key value* that represents the length of the path passing through the node. The *key values* were calculated by summing the *physical* and *estimated distances*. The physical distance is the cost from the start node to the current node. The estimated distance is the cost from the current node to the destination node. D* Lite sorts the key values of each node by inserting the discovered nodes into the priority queue. The path $p[n_0, n_k]$ generated using the D* Lite algorithm to select consecutive nodes with minimum *key values* is called the shortest evacuation path (SHP_i). Although some nodes have the same key values, this indicates that the victim's distance to the destination passing through these nodes is equal; thus, D* Lite does not enforce their order in the queue.

To increase the safety of the path generated by D* Lite, this article designed *advanced D* Lite*. *Advanced D* Lite* adopts the *danger value* as its second key. The *danger value* is the Euclidean distance between the fire source (n_{fs}) and node (n_i), and it is defined as follows:

$$\text{Danger value} = \text{Euclidean}(n_{fs}, n_i). \quad (5)$$

For Advanced D* Lite, the node with the minimum first and second key value is inserted into the safest evacuation path (SAP). The danger value makes the path as far away from the fire as possible. This decreases the changes in the computed route in response to the spread of the fire.

C. Analysis and Symbols

Table I lists the descriptions of symbols used in this paper. The time complexity of SAF is described. The time complexity of RSEC is $O(\tilde{E} \times \sim N)$, where $\tilde{E}$ denotes the number of exits and $\sim N$ denotes the number of nodes in G . Advanced D* Lite is an adaptive version of the A* algorithm that is destined for path planning in the event of an indoor fire. Each iteration of the advanced D* lite checks the victim's neighbor node until the victim reaches the destination. The time complexity of advanced D* LITE is $O(\tilde{b}^d)$, where $\tilde{b}$ is the average number of neighbor nodes and d is the path length. In summary, the time complexity of the SEPP method is $O(\tilde{E} \times \sim N + \tilde{b}^d)$. Although the time complexity of the SEPP is large, worst-case scenarios occur infrequently.

TABLE I
UNITS FOR INDOOR FIRE EVACUATION

Symbol	Description
SEPP	Safest evacuation path planning
MU	Map updater
SAF	Safest path finder
IM	Indoor floor map
RSEC	Real-time safest exit computation
IM	Indoor floor map
GM	GM is a set of identical grids which represent an indoor floor map.
g_i	Grid is represented as a square physical area within the GM.
G	Graph integrating the non-obstacle grid of the GM.
n_i	Nodes represent non-obstacle region in the Graph
n_{pass}	Boolean attribute that defines the passable property of the node
$e_k(n_i, n_j)$	Edge consecutive two adjacent nodes n_i and n_j .

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Area size	60 m $\times$ 90 m,
Grid size	1 m $\times$ 1 m;
Types of grids	5 types
Number of exits	3
Simulation rounds	500
Human moving speed	2.0 m/s

Another type of grid, colored in red, is the location of the fire source. The spread of fires with different fire sources is different, although they are in the same indoor environment. Different fire sources were simulated to evaluate the impact in Section IV-C-1 while keeping the fire source consistent for the other simulations. Only one victim was included in each simulation round. The green grid is the initial location of the victim, and it is randomly changed in each round. A simulation round ends when either the victim successfully reaches an exit, or the victim is fully surrounded by nonpassable grids.

In this article, we used the FDS [19] to simulate the spread of indoor fires. The moving speed of the victim was set at two meter per second. All simulation results were averaged over 500 simulation rounds. The parameter settings are given in Table II.

All simulations were implemented using Python programming language and run on a Windows 10 64-bit system equipped with an Intel i7-7820X CPU and 64GB RAM.

B. Evaluation Matrix

In this article, two measurements were used to evaluate the performance.

1) *Survival Rate*: *Survival rate* is the ratio of the number of successful rounds to the total number of simulation rounds. A round is successful when the victim arrives at any exit by following the planned path. It was used to assess the ability of the algorithm to find effective evacuation paths for victims. A higher *survival rate* means that the algorithm is more effective in helping victims escape fires

$$\text{Survival rate} = \text{Num}_{\text{suc}} / \text{Num}_{\text{round}}$$

where Num_{suc} is the number of simulations in which a victim successfully escaped and $\text{Num}_{\text{round}}$ is the total number of simulation rounds.

2) *Computation Time*: The *computation time* ($T_{\text{Computation}}$) consists of the *initial time* (T_{Initial}) and the *update time* (T_{Update}). It evaluates whether the algorithm satisfies the real-time constraints in fire scenarios. The *initial time* is the time to generate an evacuation path to an exit for the victim, whereas the *update time* is the time at which the path is partially adjusted in response to changes in the indoor fire scene. However, because each algorithm uses a distinct update strategy, the *number of updates* ($\text{Num}_{\text{update}}$) required also differs

$$T_{\text{Computation}} = T_{\text{Initial}} + T_{\text{Update}} * \text{Num}_{\text{update}}.$$

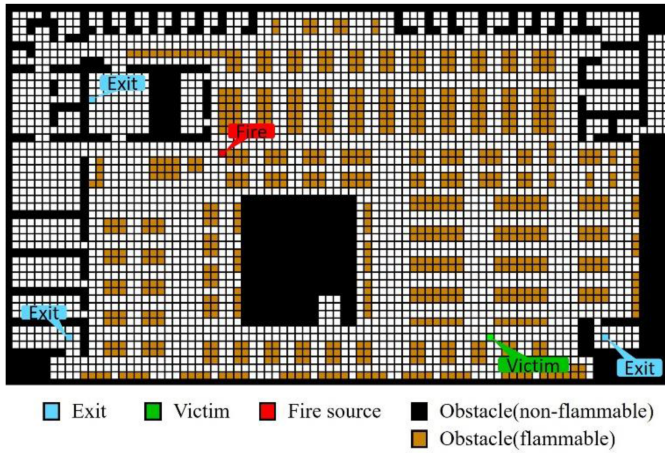


Fig. 9. Environmental field for simulations.

V. SIMULATION RESULTS AND ANALYSIS

A. Environmental Platform for Simulations

All the experiments adopted the real map of the National Cheng Kung University Library. The indoor map is a rectangle with width and length of 60 and 90 m, respectively. First, it was gridded by a 1 m $\times$ 1 m unit size, as shown in Fig. 9. There are four types of grids colored black, white, brown, and blue to describe the indoor environment. White grids are passable grids that the victim can easily cross during fires. The brown and black grids indicate the flammable and nonflammable obstacles, respectively. Victims are not allowed to pass through obstacles. Blue grids indicate exits. There were three exits in the GM. The placements of the four grid types were static during the fire simulation.

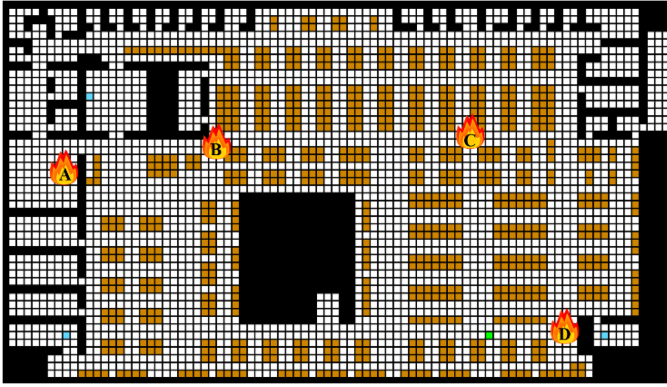


Fig. 10. Different fire source locations in indoor fire scene.

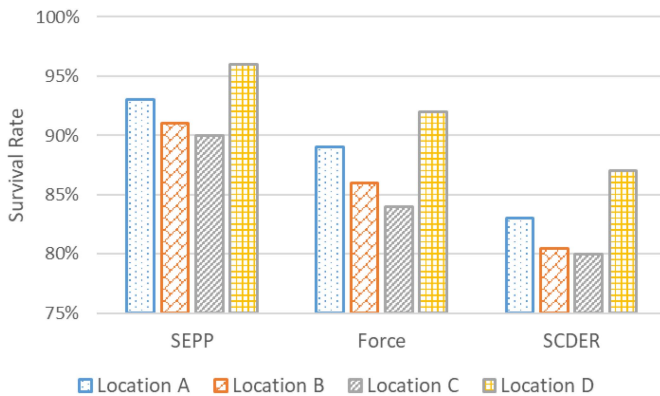


Fig. 11. Survival rate to different indoor fire scenes.

C. Experiment Results

Two other algorithms, SCDER [6] and FORCE [7], were adopted for comparison with the SEPP. SCDER periodically checks the status of each exit and uses the A* algorithm to estimate all paths from the victim's location to all safe exits. It then adopted the shortest evacuation path. FORCE transforms real-time sensing data into the concept of the resultant force and uses them to decide the safest exit periodically. The A* algorithm is then used to compute the path to the selected exit.

1) *Impact Caused by Locations of Fire Source:* To observe the impact of fire source locations, three different types of fire source locations were simulated, as illustrated in Fig. 10: *independent compartments* (A), *indoor spaces* (B and C), and *close to corners* (D).

Fig. 11 demonstrates the survival rates of the three algorithms with different fire locations. SEPP performs the best among the three algorithms, no matter the different fire sources. It is because SCDER only considers the safety of exits and the distance to decide the evacuation path, which may lead user to a dangerous area during evacuation, whereas SEPP considers fire source locations and indoor obstacles to compute the safest path. Although FORCE computes safe paths, it is time consuming to decide the exit resulting in recursively recomputations, specifically when the spread of fire is fast. The lightweight characteristic of SEPP speeds up the exit selection; thus, it can constantly update the graph with sensing data to bring it closer

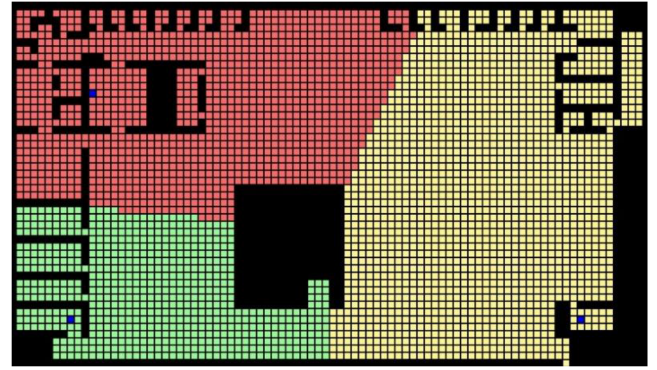


Fig. 12. Exits allocated to victims by SEPP.

to the reality of the indoor fire scene and therefore increases the survival rate.

In addition, Fig. 11 shows that when the fire source locations are in independent compartments or close to the corner, the survival rates are higher. When the fire source is in the independent compartment, the fire is affected by the nonflammable wall; therefore, the fire spread is delayed, and the victim has more time to escape. When the fire source is close to the corner, although one nearby exit becomes dangerous, the two far exits are safer, and there is a safer area to build evacuation paths for the users. On the contrary, fire sources located in open areas perform the worst because many evacuation paths are obstructed by the fire; thus, the safe paths decrease rapidly. In addition, once an indoor fire begins in an area with flammable materials, it can spread quickly and unexpectedly. To deal with the worst case, SEPP, which can rapidly adapt to changing environments, performs the best.

Fig. 12 illustrates the exit selection results of each grid calculated by our SEPP approach. It demonstrates that SEPP evenly fragments the original indoor area to three subareas. The grids with the same color are directed to the same exit. We can observe that the sizes of subareas are similar. That is, when there are multiple survivors, they will be distributed nearly evenly to exits. This is because SEPP adopts the distance between grids and exits as an impact factor to determine the selected exit. This prevents resources-contention during evacuation.

2) *Impact Caused by Area Size:* To determine how the area size impacts the computing time of different algorithms, we used different resolutions with the same real map. Taking Fig. 13 as an example, the number of grids increases from 5400 grids to 4×5400 grids by dividing each grid in a 60×90 map into four identical grids; thus, the resolution is raised to 120×180 .

As shown in Fig. 13, SEPP outperformed all other algorithms in terms of the average computation time. When the resolution of the map was magnified four times, the average computation time of the SEPP increased approximately five times, whereas that of the FORCE increased approximately 33 times. FORCE performed poorly because the critical factor affecting it is NumDanger, where NumDanger is the ratio of dangerous grids to map grids. Because the dangerous area increases as an indoor fire progresses, the computation time for the FORCE also increases significantly.

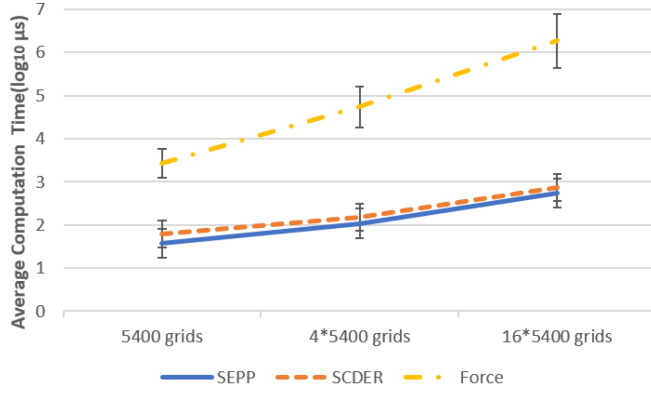


Fig. 13. Computation time to different area sizes.

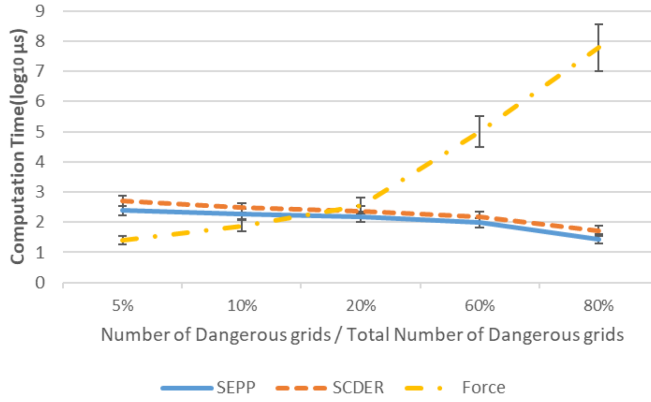


Fig. 14. Computation time to different dangerous area ratios.

The relationship between the dangerous area and computation time is illustrated in Fig. 14. The computation time for FORCE increases significantly as the size of the dangerous area increases because the amount of computation required to calculate the resultant force is proportional to the size of the dangerous area. By contrast, the critical factor affecting SEPP and SCDER is NumSafe, where NumSafe denotes the rest of the dangerous area. The fire spread rapidly from the fire source during the middle and late stages of indoor fire escape. Owing to the reduction in the number of safe grids from which victims can choose to escape, the computation time required for the SEPP and SCDER gradually decreases.

The total computation times of the three algorithms with a 16×5400 GM are given in Table III. SEPP and SCDER are more suitable for indoor fire escape applications because of their shorter average update times. Among them, SEPP only requires an average of $240 \mu s$ to compute an evacuation path. However, FORCE takes approximately 1 min to compute the escape path on a map with a resolution of 16×5400 . Nevertheless, within this minute of computation, the fire was sufficient to render the indoor scene dangerous and lethal. FORCE becomes unavailable in indoor fire scenarios as the input scale increases owing to the excessive computation time.

3) *Alternative Input of MU When Sensing Data Is Missing:* When periodic sensing data are missing, the prediction of the fire spread can be substituted as an input of the MU to update

TABLE III
COMPUTATION TIME TO 16×5400 GRIDS

	SEPP	Force	SCDER
Computation time	$2.4 * 10^2 \mu s$	$6.1 * 10^7 \mu s$	$5.2 * 10^2 \mu s$
$T_{initial}$	$5.6 * 10^{-2} \mu s$	N/A	$4.8 * 10^{-2} \mu s$
T_{update}	$1.2 \mu s$	N/A	$2.6 \mu s$
Num_{update}	198	N/A	198

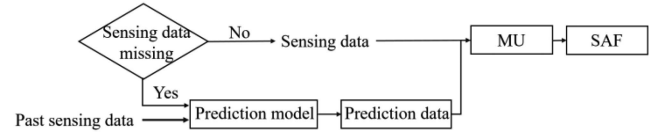


Fig. 15. Schematic diagram of the alternate prediction data when sensing data is missing.

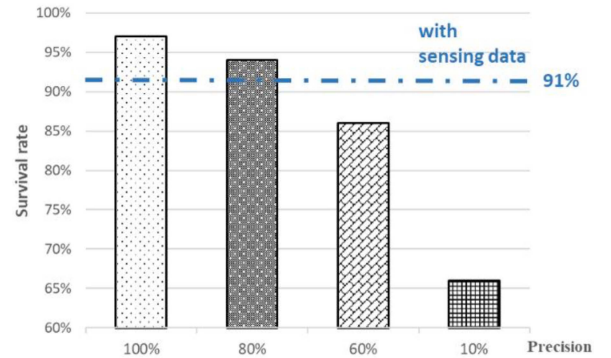


Fig. 16. Survival rate to different prediction precisions.

the graph. Fig. 15 illustrates the condition in which the sensing data are missing.

One of our fire prediction approaches was published in [22]. With the breakthrough development of artificial intelligence and impressive performance in various fields, we have applied deep learning to predict the spread of indoor fires. We proposed an long short-term memory (LSTM)-based fire-spread prediction model and conducted multiple experiments to evaluate it. The experimental results revealed that the fire prediction accuracy is approximately 0.80. The details of this process are described in an ongoing manuscript.

Additionally, we set up a simulation to evaluate the impact of different accuracies of fire spread prediction on the survival rate with SEPP. Fig. 16 illustrates that the SEPP achieves a 90% survival rate when sensing data are available. When there are alternate fire spread prediction data for missing sensing data with an accuracy of 80%, the survival rate of the SEPP is approximately 94%.

VI. CONCLUSION AND FUTURE WORKS

This article adopts a lightweight exit selection formula with the safest dynamic path planning algorithm to assist victims in escaping indoor fires. In contrast to previous indoor escape path planning methods, these methods either take too long to calculate or the output escape routes are not close to the real scene of the fire scene. Our method provides dynamic escape routes and considers the characteristics of indoor fires by choosing the safest escape route among many candidate routes. Compared with other methods, our SEPP significantly reduces the computation time and improves the survival to over 90%.

Future work will focus on two aspects. The first is to optimize the parameters of the exit selection formula using machine learning and integrate the fire prediction model into the SEPP algorithm to improve the reliability of the evacuation path. Additionally, we discovered that after some grids become dangerous, they could be converted to passable attributes at certain time points; therefore, the spread of fire changes the area affected by the fire. Therefore, another future work is to consider the movement characteristics of the fire source when planning the evacuation path.

REFERENCES

- [1] M. Dohi et al., U.S Patent No. 6,960,987, 2005.
- [2] T. Paulsen, "The effect of escape route information on mobility and way finding under smoke logged conditions," *Fire Saf. Sci.*, vol. 4, pp. 693–704, 1994.
- [3] Y. Song et al., "Fire hazard assessment with indoor spaces for evacuation route selection in building fire scenarios," *Indoor Built Environ.*, vol. 31, no. 2, pp. 452–465, 2022.
- [4] Z. Li et al., "An agent-based simulator for indoor crowd evacuation considering fire impacts," *Autom. Constructions*, vol. 120, 2020, Art. no. 103395.
- [5] F. Mirahadi and B. McCabe, "EvacuSafe: Building evacuation strategy selection using route risk index," *J. Comput. Civil Eng.*, vol. 34, no. 2, 2020, Art. no. 04019051.
- [6] J. Liu, R. Rojas-Cessa, and Z. Dong, "Sensing, calculating, and disseminating evacuating routes during an indoor fire using a sensor and diffusion network," in *Proc. 13th Int. Conf. Netw. Sens. Control*, 2016, pp. 1–6.
- [7] K. T. Ye and P. H. Tsai, "Fire simulation of building fire protection system based on digital twin," 2020, [Online]. Available: <https://hdl.handle.net/11296/8ujkix>
- [8] C. Zhang et al., "A dynamic path planning method for social robots in the home environment," *Electronics*, vol. 9, no. 7, 2020, Art. no. 1173.
- [9] H. C. Zhuang et al., "Multi-destination path planning method research of mobile robots based on goal of passing through the fewest obstacles," *Appl. Sci.*, vol. 11, no. 16, 2021, Art. no. 7378.
- [10] F. T. Yan et al., "RFES: A real-time fire evacuation system for Mobile Web3D," *Front. Inf. Technol. Electron. Eng.*, vol. 20, no. 8, pp. 1061–1074, 2019.
- [11] H. X. Jiang, "Mobile fire evacuation system for large public buildings based on artificial intelligence and IoT," *IEEE Access*, vol. 7, pp. 64101–64109, 2019, doi: [10.1109/ACCESS.2019.2915241](https://doi.org/10.1109/ACCESS.2019.2915241).
- [12] Y. Tao et al., "A mobile service robot global path planning method based on ant colony optimization and fuzzy control," *Appl. Sci.*, vol. 11, no. 8, 2021, Art. no. 3605, doi: [10.3390/app11083605](https://doi.org/10.3390/app11083605).
- [13] W. Zeng and R. L. Church, "Finding shortest paths on real road networks: The case for A," *Int. J. Geo-Graphical Inf. Sci.*, vol. 23, no. 4, pp. 531–543, 2009.
- [14] N. Adam and L. Podsedkowski, "L* algorithm—A linear computational complexity graph searching algorithm for path planning," *J. Intell. Robot. Syst.*, vol. 91, no. 3, pp. 425–444, 2018.
- [15] F. A. Raheem and I. H. Umniah, "Path planning algorithm using D* heuristic method based on PSO in dynamic environment," *Amer. Acad. Sci. Res. J. Eng. Technol. Sci.*, vol. 49, no. 1, pp. 257–271, 2018.
- [16] Y. L. Hu, X. Wang, and F.-Y. Wang, "A quantitative study of factors influence on evacuation in building fire emergencies," *IEEE Trans. Comput. Soc. Syst.*, vol. 5, no. 2, pp. 544–552, Jun. 2018, doi: [10.1109/TCSS.2018.2823869](https://doi.org/10.1109/TCSS.2018.2823869).
- [17] Z. Hong et al., "Analysis of the indoor fire risk based on the Pyrosim simulation," *Proc. IOP Conf. Ser. Earth Environ. Sci.*, vol. 636, no. 1, 2021, Art. no. 012002.
- [18] K. Sven and M. Likhachev, "D* lite," *Aaai/iaai*, vol. 15, pp. 476–483, 2002.
- [19] B. McGrattan et al., "Fire dynamics simulator—Technical reference guide," Nat. Inst. Standards Technol., Building Fire Res. Lab., 2000.
- [20] S. Koenig et al., "Lifelong planning A*," *Artif. Intell.*, vol. 155, no. 1/2, pp. 93–146, 2004, doi: [10.1016/j.artint.2003.12.001](https://doi.org/10.1016/j.artint.2003.12.001).
- [21] R. G. Tsai, Y.-Y. Tsai, and P.-H. Tsai, "Automation tool for home fire safety check," *IEEE Sens. Lett.*, vol. 5, no. 12, Dec. 2021, Art. no. 6002704, doi: [10.1109/LSSENS.2021.3124800](https://doi.org/10.1109/LSSENS.2021.3124800).
- [22] H. Y. Chen and P. H. Tsai, "A speedup simulation-based fire prediction for online evacuation planning," in *Proc. Int. Comput. Symp.*, 2020, pp. 73–78.



An-Fong Lee received the M.S. degree in science from the Institute of Manufacturing Information and Systems, National Cheng Kung University, Tainan, Taiwan, in 2022.

His research interests include localization and path planning algorithms for Internet of Things.



Pei-Hsuan Tsai (Member, IEEE) received the M.Eng. degree in computer science from Cornell University, Ithaca, NY, USA, in 2004, and the Ph.D. degree in computer science from National Tsing Hua University, Hsinchu, Taiwan, in 2010.

In 2011, she was an Assistant Professor with the Institute of Manufacturing Information and Systems, National Chen Kung University, where she is currently a Professor. Her research interest includes sensor network, data fusion, localization and path planning, hospital automation, real-time scheduling algorithms, and user interface designs. Her recent research focuses on technologies for building smart home services and digital twin system.